

Tutorial - Semana 1 - Encontro 2

Introdução

No Encontro 1 você criou o projeto do Vertical Slice e organizou sua estrutura de pastas. Neste encontro, você vai construir o primeiro elemento real de composição do projeto: um Actor com Components. Este é o conceito arquitetural mais fundamental de qualquer engine moderna, e vai aparecer, de uma forma ou de outra, em praticamente todo Blueprint que você criar pelo resto do semestre.

Objetivos da Semana

- Explicar Actor e Component como unidade universal de composição de motores, contrastando composição com herança.
- Criar um Actor Blueprint com Components via Unreal Editor.
- Implementar uma variação própria de Component, produzindo um comportamento visual diferente do demonstrado em aula.

Resultado Esperado ao Final da Semana

Um Actor Blueprint funcional no projeto do Vertical Slice, composto por múltiplos Components (malha, colisão, luz), incluindo uma variação própria criada como desafio. Combinado ao resultado do Encontro 1, a Semana 1 termina com o projeto criado, a estrutura de pastas organizada e o primeiro Actor customizado do Vertical Slice.

Pré-requisitos

- Ter concluído o Encontro 1: projeto Third Person criado e estrutura de pastas organizada.
 - Reconhecer as quatro áreas do Unreal Editor (Viewport, Content Browser, Outliner, Details).
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto criado no Encontro 1, aberto no Unreal Editor 5.6.

Arquivos necessários

- Nenhum arquivo externo.

Assets utilizados

- Formas geométricas básicas já disponíveis no projeto padrão (cubos, esferas via Static Mesh primitivo do editor). O Kenney Prototype Kit só entra a partir da Semana 3.

Projeto esperado

O mesmo projeto do Encontro 1, com a estrutura de pastas já organizada, pronto para receber o primeiro Blueprint dentro de `Blueprints/Characters` ou de uma pasta equivalente para testes iniciais.

Parte 1 — Composição sobre Herança

Objetivo

Compreender por que motores de jogos modernos organizam comportamento por composição (Components) em vez de herança de classes.

Conceito

Um problema recorrente em desenvolvimento de jogos é: como dar comportamento e forma a um objeto do mundo sem criar uma árvore rígida de heranças de classe? Se cada combinação de "tem

luz", "tem colisão", "tem malha visual", "toca som" exigisse uma nova subclasse, o número de classes explodiria rapidamente e qualquer reuso exigiria duplicar código.

A solução adotada por praticamente toda engine moderna é a **composição sobre herança**: em vez de herdar comportamento de uma classe-mãe cada vez mais específica, um objeto é montado anexando pequenas peças reutilizáveis de comportamento, chamadas de **Components**. Cada Component resolve uma responsabilidade isolada (uma malha visual, uma colisão, uma luz, um som) e pode ser reaproveitado em qualquer objeto que precise dela, sem relação hierárquica entre esses objetos.

Na Unreal, o objeto-contêiner que recebe esses Components é o **Actor** — qualquer coisa que pode existir dentro de um nível (Level) é um Actor. Um Actor sem nenhum Component é apenas um ponto no espaço; é a composição de Components que dá a ele forma, colisão, comportamento e aparência.

Passo a passo

1. No Content Browser, dentro de `Blueprints/Characters`, clicar com o botão direito e selecionar "Blueprint Class".
2. Na janela de seleção de classe-pai, escolher "Actor" (a classe-base mais genérica, sem comportamento pré-definido).
3. Nomear o novo Blueprint seguindo a convenção `BP_` (por exemplo, `BP_Testecomposicao` — este é um Actor de estudo, não um Blueprint definitivo do Vertical Slice).
4. Abrir o Blueprint recém-criado com um duplo clique.

Resultado esperado

O editor de Blueprint abre em uma janela própria, mostrando um Actor vazio, sem nenhum Component além do componente raiz padrão (DefaultSceneRoot).

Verificando

Confirme, no painel de Components (à esquerda do editor de Blueprint), que existe apenas o componente raiz e nenhum outro elemento visual ou de colisão.

Problemas comuns

- **Selecionar "Character" em vez de "Actor"**: Character já vem com Components de movimentação pré-configurados, o que esconderia o processo de composição manual que queremos entender aqui. Para este exercício, a classe-pai correta é Actor puro.
- **Esquecer de nomear com o prefixo `BP_`**: o Blueprint segue com o nome padrão `NewBlueprint`, o que viola a convenção de nomenclatura do projeto (ver

Boas práticas

Mantenha um Actor puro (sem herança de Character ou Pawn) como ponto de partida sempre que quiser entender ou ensinar composição do zero — subclasses mais especializadas já escondem Components padrão que dificultam a visualização do processo.

Comparação com Unity

O Actor da Unreal corresponde ao GameObject da Unity. Assim como um Actor vazio é apenas um ponto no espaço até receber Components, um GameObject vazio na Unity é apenas um Transform até receber Components (como MeshRenderer, Collider, AudioSource). O princípio de composição é idêntico — a diferença é que o GameObject nasce como um contêiner genérico desde a raiz da engine, enquanto o Actor da Unreal é uma classe com ciclo de vida próprio, sobre a qual os Components são anexados.

Parte 2 — Anexando Components

Objetivo

Anexar Components de malha, colisão e luz a um Actor, entendendo a responsabilidade de cada um.

Conceito

Cada Component representa uma responsabilidade isolada e bem definida:

- Um **Static Mesh Component** dá forma visual tridimensional ao Actor (a malha que é renderizada).
- Um **Collision Component** (ou a colisão embutida no próprio Static Mesh) define como o Actor interage fisicamente com outros objetos do mundo — se pode ser atravessado, se bloqueia o jogador, se dispara eventos de sobreposição.
- Um **Light Component** (por exemplo, Point Light) faz o Actor emitir luz no ambiente.

Nenhum desses Components sabe da existência dos outros. Essa independência é o que permite reaproveitar o mesmo Static Mesh Component em um Actor totalmente diferente de onde se reaproveita um Point Light Component — cada peça resolve seu próprio problema, isoladamente.

Imagem sugerida

Objetivo: mostrar o painel de Components do Blueprint com três Components anexados (Static Mesh, Point Light, colisão). Enquadramento: captura do editor de Blueprint com o painel de Components à esquerda expandido. Elementos importantes: hierarquia mostrando DefaultSceneRoot como pai dos três Components anexados. O que deve ser destacado: a estrutura em árvore do painel de Components, com cada item rotulado por sua função. Legenda sugerida: "Um Actor composto por três Components independentes: malha, colisão e luz." Referência visual (documentação oficial, apenas para consulta — não copiar a imagem): [Components in Unreal Engine](#)

Passo a passo

1. No editor de Blueprint aberto, no painel de Components, clicar em "Add Component".
2. Selecionar "Static Mesh" e, no painel Details, atribuir uma malha simples (por exemplo, o cubo padrão do editor, `Shape_Cube` ou equivalente disponível no projeto).
3. Clicar em "Add Component" novamente e selecionar "Point Light".
4. No painel Details do Point Light, ajustar a posição (Transform) para que a luz fique visivelmente acima ou ao lado da malha.
5. Verificar, no painel Details do Static Mesh, se a colisão já está habilitada por padrão (a maioria das malhas primitivas já vem com colisão simples configurada); caso não esteja, ativar "Generate Overlap Events" ou o preset de colisão apropriado.
6. Compilar o Blueprint (botão "Compile" no topo do editor).
7. Salvar o Blueprint.
8. Arrastar uma instância do `BP_TesteComposicao` do Content Browser para dentro do Viewport, posicionando-a próxima ao personagem do template.
9. Testar em modo Play (Play in Editor) e observar o Actor no mundo: sua malha visível, a luz emitida e a colisão bloqueando o personagem.

Resultado esperado

O Actor aparece no nível com forma visual (o cubo ou malha escolhida), emitindo luz ao redor, e bloqueia fisicamente o personagem do jogador ao colidir com ele em modo Play.

Verificando

No modo Play, caminhe com o personagem até encostar no Actor recém-criado. Confirme que o personagem é bloqueado (não atravessa a malha) e que a iluminação da cena muda visivelmente perto do Point Light.

Problemas comuns

- **Luz não aparece:** verifique se a intensidade (Intensity) do Point Light não está em zero, e se a construção de iluminação (Build Lighting) não é necessária para luzes dinâmicas — Point Lights móveis costumam funcionar em tempo real sem rebuild.
- **Personagem atravessa o Actor:** confira o preset de colisão do Static Mesh Component no painel Details; ele deve estar como "BlockAll" ou equivalente, não "NoCollision".
- **Esquecer de compilar antes de testar:** alterações no Blueprint só têm efeito no Viewport e no Play após clicar em "Compile".

Boas práticas

Nomeie cada Component de forma descritiva no próprio painel de Components (por exemplo, `MalhaPrincipal`, `LuzAmbiente`) em vez de manter os nomes genéricos gerados automaticamente (`StaticMesh1`, `PointLight1`) — isso facilita a leitura do Blueprint por qualquer pessoa, incluindo você mesmo em semanas futuras.

Comparação com Unity

Anexar um Static Mesh Component, um Collision Component e um Point Light Component a um Actor na Unreal é equivalente a anexar um MeshRenderer + MeshFilter, um Collider e uma Light a um GameObject na Unity. O fluxo mental é o mesmo: cada peça de comportamento é um Component independente, anexado ao mesmo contêiner. A diferença de implementação é que a Unreal frequentemente combina malha e colisão em um único Static Mesh Component (a colisão pode vir embutida na malha importada), enquanto a Unity trata MeshFilter/MeshRenderer e Collider como três elementos sempre separados.

Parte 3 — Desafio de Variação

Objetivo

Aplicar o conceito de composição de forma autônoma, adicionando um Component diferente do demonstrado.

Conceito

Esta etapa não introduz um conceito novo — ela testa se o conceito de composição foi de fato compreendido, pedindo que você resolva um pequeno problema de variação sem um passo a passo fechado. É a primeira manifestação, ainda mínima, da autonomia que vai crescer ao longo do semestre (ver PEDAGOGICAL_RULES.txt: a orientação diminui continuamente a partir do Módulo 1).

Passo a passo

1. Reabrir o `BP_TestComposicao` (ou criar uma variação em um novo Blueprint, se preferir manter o original como referência).
2. Adicionar um Component adicional que produza um comportamento visual diferente do demonstrado — por exemplo, uma segunda malha em outra posição, uma luz de cor diferente, ou um Component de rotação simples (Rotating Movement Component) aplicado à malha.
3. Configurar as propriedades desse novo Component no painel Details até obter o efeito visual desejado.
4. Compilar, salvar e testar em modo Play.

Resultado esperado

Um comportamento visual perceptivelmente diferente do Actor demonstrado nas Partes 1 e 2, resultante da adição de pelo menos um Component extra.

Verificando

Compare seu resultado com o de um colega: as soluções devem ser visivelmente diferentes entre si, já que o desafio não tem gabarito único.

Problemas comuns

- **Criar uma nova classe de Actor em vez de adicionar um Component:** é comum tentar resolver a variação criando uma subclasse nova, reproduzindo o padrão de herança de programação tradicional. Relembre o princípio de composição sobre herança antes de prosseguir — a solução esperada é anexar um Component ao Actor já existente.

Boas práticas

Documente rapidamente (em um comentário no próprio Blueprint, usando uma Comment Box) qual Component você adicionou e por quê — esse hábito de comentar pontos de decisão será cobrado formalmente a partir do Módulo 2, no Code Review.

Comparação com Unity

A mesma variação, na Unity, seria resolvida anexando um novo Component (por exemplo, um script customizado de rotação, ou um segundo MeshRenderer em um GameObject filho) ao GameObject existente — nunca criando uma nova classe C# que herda de outro MonoBehaviour para esse fim. O princípio de resolver variação por composição, não por herança, é o mesmo nas duas engines.

Ao final da semana

Ao final da Semana 1 (Encontros 1 e 2), o projeto deve conter: a estrutura de pastas completa do Vertical Slice (Encontro 1) e um Actor Blueprint funcional composto por múltiplos Components, incluindo a variação própria do desafio (Encontro 2). Isso corresponde ao início da linha "Actor + Component base" do roadmap do Módulo 1 no PROJECT_ARCHITECTURE.md — a base sobre a qual o BP_Player será construído a partir da Semana 2.

Desafio

Além da variação já pedida na Parte 3, experimente anexar dois Components adicionais de tipos diferentes dos já usados (por exemplo, um Audio Component com um som de teste, ou um segundo Static Mesh Component posicionado como um "detalhe" do objeto) e observe como o Actor cresce em complexidade sem exigir nenhuma nova classe — apenas novos Components no mesmo Actor.

Checklist

- ☐ BP_Testecomposicao criado como subclasse de Actor (não de Character ou Pawn)
- ☐ Actor possui ao menos um Static Mesh Component, um Point Light Component e colisão funcional
- ☐ Personagem do jogador é bloqueado ao colidir com o Actor em modo Play
- ☐ Componente adicional do desafio produz uma variação visual própria, diferente do demonstrado

☐ Blueprint compilado sem erros e salvo

☐ Nomes dos Components e do Blueprint seguem a convenção `BP_` do projeto

Glossário

- **Actor:** unidade básica de qualquer objeto que pode existir em um nível da Unreal; um contêiner que recebe Components.
- **Component:** peça reutilizável de comportamento ou forma, anexada a um Actor (malha, colisão, luz, som, lógica).
- **Composição:** padrão arquitetural em que comportamento é montado combinando peças independentes (Components), em vez de herdar de uma classe cada vez mais específica.
- **Herança:** padrão em que uma classe reutiliza e especializa o comportamento de uma classe-mãe; contrastado aqui com composição.
- **Static Mesh Component:** Component responsável pela malha visual tridimensional de um Actor.
- **Point Light Component:** Component responsável por emitir luz a partir de um ponto no espaço.

Referências

- EPIC GAMES. **Actors in Unreal Engine**. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/actors-in-unreal-engine>.
- EPIC GAMES. **Components in Unreal Engine**. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/components-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — Gameplay Framework (Actors e Components). Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- EPIC GAMES. **Blueprints Visual Scripting**. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine>.
- UNITY TECHNOLOGIES. **Unity Manual** — GameObjects e Components, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**.